

Final Project Report

[TODO: Topic 1 / 2 / 3 / 4 - title]

Team: [TODO: Your team name]

Repo: [TODO: GitHub repo URL]

Members: Member A name (@github-A),
Member B name (@github-B),
Member C name (@github-C)

Date: [TODO: Date submitted]

Tag: v1.0-final

Executive Summary

Required. Exactly one paragraph (5–7 sentences). What you built, the headline concurrency speedup you measured, your headline robustness story, and the single most surprising thing the team learned. *This is the first thing the grader reads. If you cannot write it clearly, you do not yet understand your own project.*

Your paragraph should answer: What problem did the system solve? What is the one number that proves the concurrency design works? What is the one failure mode you handled well? What would you change if you started over?

[Your response here – replace this placeholder.]

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	3
2 Architecture	3
2.1 Module map	3
2.2 OOP design & key abstractions	4
2.3 Data flow across module boundaries	4
3 Concurrency & Performance	5
3.1 Why this concurrency model	5
3.2 Sequential-vs-concurrent benchmark	5
3.3 Rate limit & politeness	6

4	Robustness & Error Handling	6
4.1	Wrapping the AI module	6
4.2	Failure-mode analysis (one concrete incident)	7
4.3	Input validation	7
5	Storage & Caching	7
5.1	Schema	7
5.2	Caching policy	8
6	Testing	8
6.1	Strategy & coverage	8
6.2	Notable tests	8
7	Deployment & Reproducibility	9
7.1	Docker image	9
7.2	Configuration	9
8	Limitations & Future Work	9
9	Tools & Acknowledgements	10
	References	10
A	Appendix — Reproducing the benchmark	10

1. Project Overview

1.1 Problem framing & scope

Required. 2–3 paragraphs. State the topic problem in your own words. List the inputs, outputs, and any user-facing entry points (CLI commands, HTTP endpoints). State which provider stack you used (LLM, embedding, USDA, web search).

Beyond the obvious: which parts of the TOPIC.md spec did your team explicitly decide *not* to do, and why? Where did you tighten the spec? Where did you loosen it?

[Your response here – replace this placeholder.]

1.2 Provider choice and the AI module contract

Required. Name the LLM, embedding, and any auxiliary provider you used, and the model identifiers (`claude-sonnet-4-6`, `text-embedding-3-small`, etc.). State what `ai/` functions you call. *Confirm in one sentence that you did not modify the public interface of the provided `ai/` package.*

Push deeper: how does your code defend against a malformed model response? What happens when the provider returns valid JSON that is semantically wrong (wrong topic enum, negative grams, etc.)?

[Your response here – replace this placeholder.]

2. Architecture

2.1 Module map

Required. An architecture diagram (boxes-and-arrows or sequence diagram) showing the main modules and their dependencies. Identify on the diagram: the boundary of the provided `ai/` package, your service layer that wraps it, your core business logic, your concurrency layer, your storage layer, and your CLI/HTTP entry points.

Interpret your own diagram: which arrows cross the AI-module boundary? Which arrows cross the storage boundary? If you had to swap providers (Anthropic → OpenAI), how many files would change?

[Replace this box with `\includegraphics{...}` or a `tikzpicture`.]

Module-by-module summary (1–2 sentences each):

[Your response here – replace this placeholder.]

2.2 OOP design & key abstractions

Required. Identify at least one example of inheritance or composition *in your own code* (not the provided package's). Quote 5–10 lines of the relevant class or object design. Justify why this abstraction was the right tool; if you used an ABC, explain why.

Defensibility: if a grader asked "why is this an abstract base class rather than a Protocol, a plain function with a callable parameter, or a configuration enum?" — what is your answer? Are there places where you used composition where inheritance would have been wrong, or vice versa?

Code listing:

```
1 # Replace with your own ABC + concrete subclass example.
2 class StorageBackend(ABC):
3     @abstractmethod
4     def save(self, item: Item) -> None: ...
5
6 class SQLiteStorage(StorageBackend): ...
```

Justification (2–3 sentences):

[Your response here – replace this placeholder.]

2.3 Data flow across module boundaries

Required. List the dataclasses you defined for your own SE layer. Confirm in one sentence that no naked dictionaries cross module boundaries.

The hardest call: when did you reuse a dataclass from the provided `ai/` package vs. wrap it in a new one for your SE layer? What was the trigger for wrapping?

[Your response here – replace this placeholder.]

3. Concurrency & Performance

3.1 Why this concurrency model

Required. State which concurrency primitive you used (`asyncio`, `ProcessPoolExecutor`, `ThreadPoolExecutor`) and why. Identify the bottleneck the parallel design targets. State the bound (e.g. `Semaphore(10)`) and how you picked it.

Justify against alternatives: could you have used threads instead of `asyncio`? Why not? What is the smallest workload at which the concurrent design starts to win against sequential? What happens if you set the semaphore to 1 vs. 100?

[Your response here – replace this placeholder.]

3.2 Sequential-vs-concurrent benchmark

Required. A table reporting wall-clock time for the same workload run sequentially vs. concurrently. State N (number of requests), the machine, the python version, whether caches were cleared, and the command line. Reproduce the table also in your README.

Honest interpretation: did you reach the theoretical $N \times$ speedup? If not, what was the new bottleneck (provider rate limit? GIL? network RTT? single-threaded DB writes?). What would push the curve further?

Workload	N	Sequential (s)	Concurrent (s)	Speedup
<i>e.g. register 20 items</i>	_____	_____	_____	_____
<i>e.g. analyze 16 meals</i>	_____	_____	_____	_____

Discussion (2–3 sentences):

[Your response here – replace this placeholder.]

3.3 Rate limit & politeness

Required. State the rate-limit strategy (sleep-on-429, token bucket, leaky bucket, request budget). Document the configured budget. State the highest burst your test suite produces.

Failure-mode test: have you actually observed a 429 from the provider during development, or is this strategy untested in anger? What does the log look like when the budget is hit?

[Your response here – replace this placeholder.]

4. Robustness & Error Handling

4.1 Wrapping the AI module

Required. Describe your service-layer wrapper around `ai.*` calls: retries (which exceptions, how many attempts, backoff schedule), timeouts (per-call value), structured logging (what fields, at what level), input validation (what you check before calling, what you check on the response).

Pressure-test: what happens if the provider returns a 500? A 429? A connection reset mid-stream? A perfectly-formed JSON that contradicts the schema? Walk through one of these end-to-end.

[Your response here – replace this placeholder.]

4.2 Failure-mode analysis (one concrete incident)

Required. Pick *one* failure mode you actually exercised (provider 5xx, timeout, malformed response, network drop, disk full, corrupted image, USDA outage, etc.). Describe: (i) how you injected the failure, (ii) what the system did, (iii) what the user saw, (iv) what the logs showed.

Go beyond "graceful degradation worked": what did you originally expect to happen vs. what actually happened? Did the test surface any design issue that you then fixed? Did the log give you enough to diagnose it without running the code again?

[Your response here – replace this placeholder.]

4.3 Input validation

Required. For every entry point (CLI, HTTP, file, env), list the validation rules. State what error message the user gets when validation fails.

Adversarial: could a malformed image kill your service? A 100 MB JSON request? A query of 50,000 characters? A path that escapes your storage directory?

[Your response here – replace this placeholder.]

5. Storage & Caching

5.1 Schema

Required. The SQLite schema for your domain tables. Include indices and foreign keys. State which fields are derived (cached) vs. source-of-truth.

Schema choices: why did you store embedding bytes as a blob rather than serialized JSON, or vice versa? What happens to the DB if the embedding dimension changes (a different provider, a different model)?

-- Paste your schema here, e.g.

```
-- CREATE TABLE items (
--   id INTEGER PRIMARY KEY AUTOINCREMENT,
--   status TEXT NOT NULL CHECK(status IN ('lost', 'found')),
--   user_text TEXT NOT NULL,
--   vlm_description TEXT NOT NULL,
--   embedding BLOB NOT NULL,
--   created_at TEXT NOT NULL
-- );
```

5.2 Caching policy

Required. What do you cache, how long, where, and how is the cache invalidated. Report cache hit rate from a demo run (a single number is fine).

Correctness vs. freshness: which cached values can become stale (e.g. USDA nutrition for a renamed food, an LLM summary if the prompt changes)? How would you detect and evict them in production?

[Your response here – replace this placeholder.]

6. Testing

6.1 Strategy & coverage

Required. Report the coverage number (`pytest -cov`). State how you mocked the AI module and the HTTP layer. Confirm the provided `tests/test_ai_smoke.py` still passes.

Quality vs. coverage: which lines did you decide *not* to cover, and why? Which mocks did you have to redesign mid-project because they were testing the wrong thing?

Suite	Coverage
src/ (your SE layer)	_____ %
Provided ai/ smoke tests	_____ <i>passing?</i> <i>yes</i> / <i>no</i>

6.2 Notable tests

Required. Highlight at least three tests: (i) one happy-path end-to-end test, (ii) one concurrency test (e.g. that `asyncio.gather` behaves correctly when one task raises), (iii) one error-path test that surfaces a real bug you would otherwise have missed.

What didn't you test? What is the test you wish you had time to write?

[Your response here – replace this placeholder.]

7. Deployment & Reproducibility

7.1 Docker image

Required. The exact `docker build` and `docker run` commands. The image size. The Python version baked in. Note whether you use a single-stage or multi-stage build.

Engineering trade-offs: did you ship an Alpine, slim, or full base image, and why? How does your image size compare to a fresh `python:3.12-slim` (~120 MB)?

[Your response here – replace this placeholder.]

7.2 Configuration

Required. A short list of every environment variable the app reads, and what happens if each is missing.

[Your response here – replace this placeholder.]

8. Limitations & Future Work

Required. 3–5 bullet points. Honest list of where the system is brittle, what it would take to productionize, and what you would do differently with another week.

Be honest: no project is perfect. What was the worst design decision you made? What did you not get to test under load? What is the scenario most likely to break the system on a busy day?

[Your response here – replace this placeholder.]

9. Tools & Acknowledgements

Required. Disclose substantial AI-assistant use. For each significant LLM-aided contribution: which module, which assistant (Claude/ChatGPT/Cursor/Copilot/etc.), and whether the team reviewed and adapted it.

Cultural note: "AI helped with everything" is not a useful disclosure; "Cursor drafted the initial `retry_policy.py`; we rewrote backoff logic after observing it hit the rate limit" is.

[Your response here – replace this placeholder.]

References

- [1] [TODO: e.g. Anthropic API documentation, <https://docs.anthropic.com/...>]
- [2] [TODO: e.g. tenacity library: <https://tenacity.readthedocs.io/>]
- [3] [TODO: e.g. FastAPI tutorial / pydantic-settings docs]
- [4] [TODO: Any paper, blog post, or talk that shaped a design decision.]

A. Appendix — Reproducing the benchmark

Exact commands to reproduce the §3 benchmark. Anyone with the repo and a valid `.env` should be able to run the lines below and get numbers within 20% of the ones in the table.

```
# Replace with your exact reproducer:
$ git clone <repo> && cd <repo>
$ cp .env.example .env # fill in keys
$ docker build -t finalproj .
$ docker run --env-file .env finalproj python scripts/bench.py --N 20
```

End of report — thank you for reading.